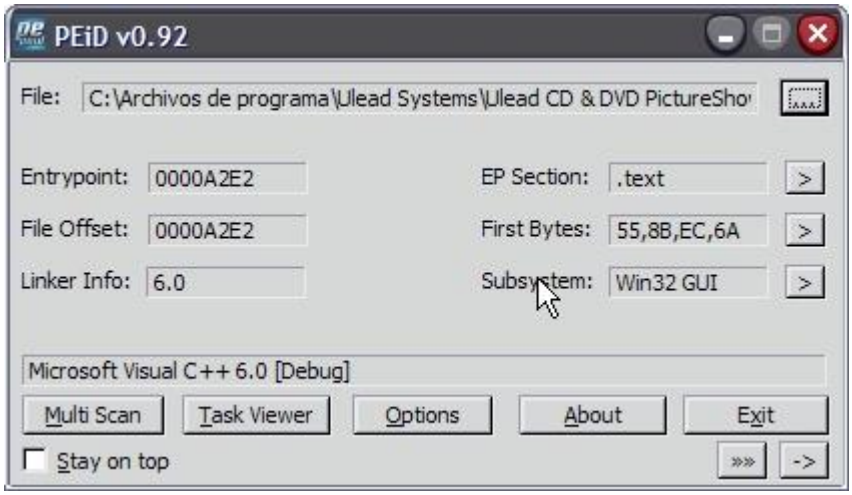
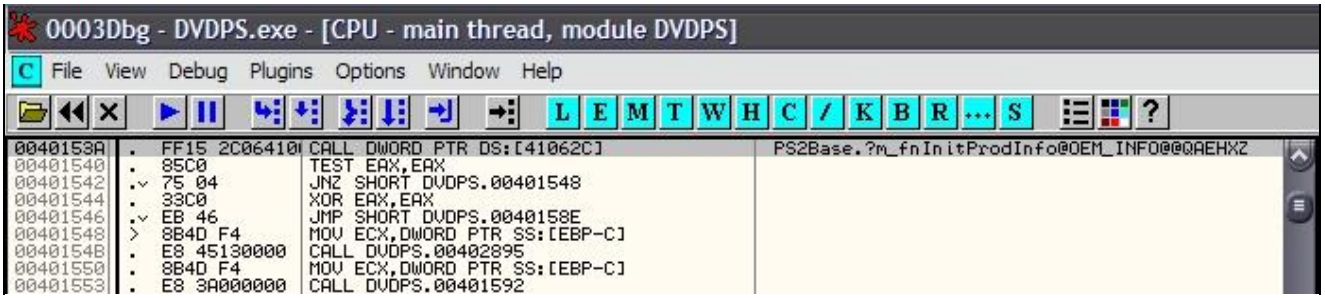
Programa:	Ulead CD & DVD PictureShow 3 Trial	
Protección:	Trial 14 días	
Objetivos:	Evitar que expire	
URL descarga:	www.ulead.com	
Dificultad:	Fácil	
Herramientas:	OllyDbg + PEiD	
Cracker:	SyXe'05 [hAcK-c0d3d]	Fecha: 07-08-2005

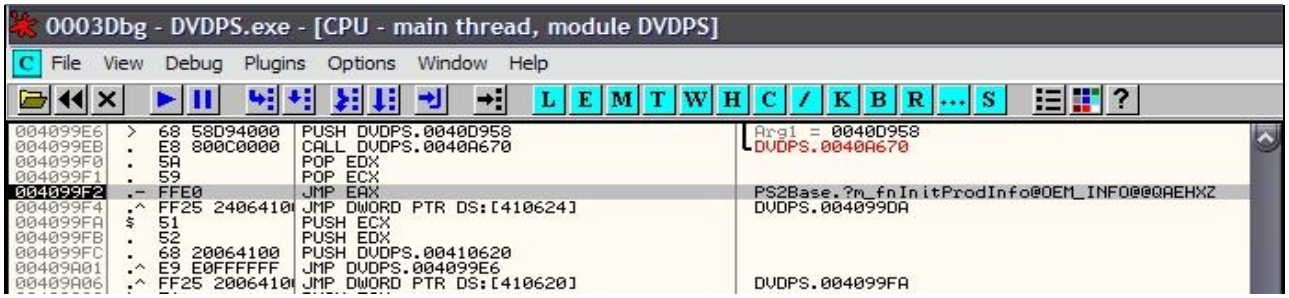
Hola de nuevo a todos, aquí estamos con otra entrega, esta vez la víctima es un otro producto de Ulead, esta vez le toca el turno a ***Ulead CD & DVD PictureShow 3 Trial*** como siempre lo cargaremos en el PEiD a ver que información nos da acerca del ejecutable:



Visual C++ ¿? vaya, creí que estaría empaquetado, pues mejor para nosotros, primeramente haremos una primera pasada de reconocimiento en ***Olly*** a ver que tal, traceando con ***F8*** de momento a ver qué hace y donde, de momento no necesitamos entrar en los calls, llegamos a esta parte:



El nombre es muy llamativo, saltará a la función ***fnInitProdInfo***... donde parece ser que va a extraer información de inicialización del producto, entremos dentro con ***F7***...

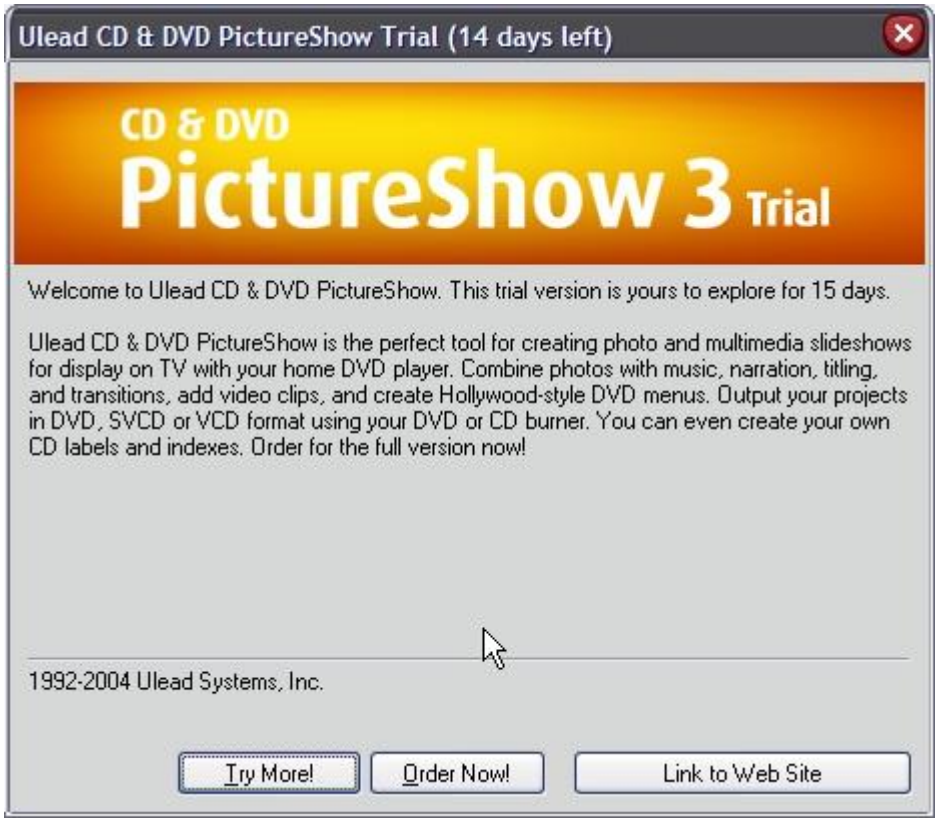


Ese ***JMP EAX*** nos pone de pleno en la librería ***PS2Base.dll*** que tiene en su mismo directorio y lógicamente es propia del programa y no común a todos, entonces si encontráramos algo en ella sí podríamos parchear ya que

sólo afectará al comportamiento de este programa concreto y no a los demás ya que nadie utiliza esta ***dll***, sigamos con el traceo suave y con ***F8*** (salvo que veamos nombres relevantes como este anterior), atención aquí:

```
0003Dbg - DVDPS.exe - [CPU - main thread, module PS2Base]
File View Debug Plugins Options Window Help
017B4223   E8 10060000    CALL PS2Base.?m_fnGetProductVersion@OEM_INFO@@QAE?AW4ProductVersion@1@XZ
017B4228   8985 E8FEFFFF  MOV DWORD PTR SS:[EBP-118],EAX
017B422E   83BD E8FEFFFF  CMP DWORD PTR SS:[EBP-118],1
017B4235   0F85 A3020000  JNZ PS2Base.017B44DE
017B423B   8B85 E4FEFFFF  MOV EAX,DWORD PTR SS:[EBP-11C]
017B4241   83C0 08        ADD EAX,8
017B4244   50             PUSH EAX
```

En este ***call fnGetProductVersion***.. sale la ***nag*** principal del programa en pantalla que es esta:



Como acabamos de instalar el producto nos dice que tenemos ***14*** días de prueba por delante (***days left***), no veas cada vez nos dan menos días, llegará un punto en que nos van a dar ***12 hours left***, jeje.. pero esto a nosotros no nos importa mucho ya que no nos afecta ***;XDD***

Aceptaremos pulsando en ***Try More!*** retornamos con ***EAX=1*** interesante dato, lo anotamos, además guarda este valor de retorno en ***[EBP-118]*** que luego compara con 1 y si son iguales salta en ***:17B422E*** hacia una zona muy alejada ;) esto me gusta, es el típico esquema ***naggeriano*** donde se comprueba si te queda tiempo ***Trial***, en caso de que no pues eso un ***EAX=0*** como una casa y a dormir, si ***EAX=1*** puedes disfrutar del producto, a veces la asignación es al revés y ***EAX=0*** puedes disfrutar y ***EAX=1*** a la calle (lógica inversa) de momento sigamos con el traceo, recordad que estamos sólo reconociendo el terreno que pisamos, ya habrá tiempo de atacar después... ah! no cal que diga que debemos poner un ***BPX*** en el ***call_nag*** verdad? venga ponerlo los que no lo hayáis hecho, jeje. y ahora sigamos..

No hay que tracear mucho para llegar a la salida de la rutina en la que estamos, recordad que ahora mismo nos encontramos en una librería llamada ***PS2Base.dll*** que como sabemos está en la misma carpeta donde se instaló el programa, ir a mirarlo el que quiera y allí la verá justo al lado de ***PS2BaseRC.dll***, pero esta de momento nada, como veníamos diciendo no hemos traceado mucho más para encontrar este salto incondicional:

```
0003Dbg - DVDPS.exe - [CPU - main thread, module PS2Base]
017B44CE   8B45 FC        MOV EAX,DWORD PTR SS:[EBP-4]
017B44D1   8B10           MOV EDX,DWORD PTR DS:[EAX]
017B44D3   8B4D FC        MOV ECX,DWORD PTR SS:[EBP-4]
017B44D6   FF52 10        CALL DWORD PTR DS:[EDX+10]
017B44D9   E9 B1020000    JMP PS2Base.017B478F
017B44DE   83BD E8FEFFFF  CMP DWORD PTR SS:[EBP-118],0
017B44E5   0F85 A0020000  JNZ PS2Base.017B478B
017B44EB   8B85 E4FEFFFF  MOV EAX,DWORD PTR SS:[EBP-11C]
017B44F1   83C0 08        ADD EAX,8
```

seguimos el salto y nos deja aquí:

```
0003Dbg - DVDPS.exe - [CPU - main thread, module PS2Base]
017B4786   FF52 10        CALL DWORD PTR DS:[EDX+10]
017B4789   EB 04          JMP SHORT PS2Base.017B478F
017B478B   33C0           XOR EAX,EAX
017B478D   EB 1B          JMP SHORT PS2Base.017B47AA
017B478F   8B45 FC        MOV EAX,DWORD PTR SS:[EBP-4]
017B4792   8B10           MOV EDX,DWORD PTR DS:[EAX]
017B4794   8B4D FC        MOV ECX,DWORD PTR SS:[EBP-4]
```

Allí remarqué lo que me llamó la atención y es que pasmos por encima de un '***XOR EAX, EAX***' muy significativo y por el contrario nos pone sobre un ***MOV EAX, [EBP-4]*** y pero esto no es todo, mirad esto:

```
0003Dbg - DVDPS.exe - [CPU - main thread, module PS2Base]
017B4786   FF52 10        CALL DWORD PTR DS:[EDX+10]
017B4789   EB 04          JMP SHORT PS2Base.017B478F
017B478B   33C0           XOR EAX,EAX
017B478D   EB 1B          JMP SHORT PS2Base.017B47AA
017B478F   8B45 FC        MOV EAX,DWORD PTR SS:[EBP-4]
017B4792   8B10           MOV EDX,DWORD PTR DS:[EAX]
017B4794   8B4D FC        MOV ECX,DWORD PTR SS:[EBP-4]
017B4797   FF52 04        CALL DWORD PTR DS:[EDX+4]
017B479A   8B45 FC        MOV EAX,DWORD PTR SS:[EBP-4]
017B479D   8B10           MOV EDX,DWORD PTR DS:[EAX]
017B479F   8B4D FC        MOV ECX,DWORD PTR SS:[EBP-4]
017B47A2   FF52 14        CALL DWORD PTR DS:[EDX+14]
017B47A5   B8 01000000    MOV EAX,1
017B47AA   8BE5           MOV ESP,EBP
017B47AC   5D             POP EBP
017B47AD   C3             RETN
```

Pues más claro el agua, tenemos un call con dos posibles valores de retorno: ***EAX=1*** ó ***EAX=0***,y lo más interesante es que en este call se muestra la nag lo que indica la importancia del mismo, de momento seguiremos como si nada a ver, superemos el retorno para salir en la zona de la llamada a la dll:

```
0003Dbg - DVDPS.exe - [CPU - main thread, module DVDPS]
00401534   .  81C1 4C02000(ADD ECX,24C
0040153A   .  FF15 2C06410(CALL DWORD PTR DS:[41062C]
00401540   .  85C0         TEST EAX,EAX
00401542   .v 75 04        JNZ SHORT DVDPS.00401548
00401544   .  33C0         XOR EAX,EAX
00401546   .v EB 46        JMP SHORT DVDPS.0040158E
00401548   >  8B4D F4      MOV ECX,DWORD PTR SS:[EBP-C]
0040154B   .  E8 45130000  CALL DVDPS.00402895
00401550   .  8B4D F4      MOV ECX,DWORD PTR SS:[EBP-C]
```

Si miramos los registros vemos ***EAX=1*** que es el valor que nos devolvió el call luego vemos un ***TEST EAX, EAX*** que mira precisamente si ***EAX*** vale ***0*** y luego un ***JNZ*** que saltará en caso de ***EAX=!0***, esta parte es importante porque si no se produce el salto nos comemos un ***XOR EAX, EAX*** y un ***JMP*** que nos aleja bastante, lo he comprobado y nos lleva a ***Exit_Process()*** o sea que ya tenemos algo: El call anterior debe retornar ***EAX=!0***, un ***EAX=1*** ya nos viene bien, y justamente ese call es el encargado de mostrar la nag inicial y todo, o es muy fácil esto o es que se ve muy claro, pongamos un ***BPX*** en ***:40153A*** y reiniciemos con ***Ctrl+F2***, le damos a ***F9*** para

parar allí, entremos de nuevo pero ahora ya dispuestos a todo ;XDDD cuando pare entraremos, llegaremos al ***JMP EAX***, para adentro pero ahora pasaremos en la llamada ***call fnGetProductVersion*** a ver que hay en el interior:

```
0003Dbg - DVDPS.exe - [CPU - main thread, module PS2Base]
File View Debug Plugins Options Window Help

017B4852   6A 00          PUSH 0
017B4854   E8 05B40100    CALL <JMP.&u32Cfg.GetModuleInfo>
017B4859   83C4 10        ADD ESP,10
017B485C   8B4D AC        MOV ECX,DWORD PTR SS:[EBP-54]
017B485F   898D 3CFEFFFF  MOV DWORD PTR SS:[EBP-1C4],ECX
017B4865   81BD 3CFEFFFF  CMP DWORD PTR SS:[EBP-1C4],515
017B486F   74 11          JE SHORT PS2Base.017B4882
017B4871   81BD 3CFEFFFF  CMP DWORD PTR SS:[EBP-1C4],579
017B487B   74 25          JE SHORT PS2Base.017B48A2
017B487D   E9 83000000    JMP PS2Base.017B4905
017B4882   8D95 48FEFFFF  LEA EDX,DWORD PTR SS:[EBP-1B8]
017B4888   52             PUSH EDX
017B4889   6A 00          PUSH 0
017B488B   E8 C8B30100    CALL <JMP.&u32Cfg.ulcCheckLegality>
017B4890   85C0           TEST EAX,EAX
017B4892   75 07          JNZ SHORT PS2Base.017B489B
017B4894   B8 04000000    MOV EAX,4
017B4899   EB 6C          JMP SHORT PS2Base.017B4907
017B489B   B8 01000000    MOV EAX,1
017B48A0   EB 65          JMP SHORT PS2Base.017B4907
```

La nag aparece en ***:17B488B*** donde se llama a la función ***ulcCheckLegality***, jeje.. vaya nombre no? no dejan de sorprenderme, bueno en este call pinta la nag en pantalla, poco antes hay tres saltos de los cuales uno de ellos pasa por encima del call, ponemos un ***BPX*** en la zona pero ***Olly*** no para, no hay problema llegaremos traceando desde el ***JMP EAX*** que no hay mucho trozo, allí nos paramos en ***:17B4865*** y hacemos un ***Follow in Dump***:

```
Address   Hex dump                                          ASCII
0012FC04  15 05 00 00 44 09 41 00 00 00 00 00 55 6C 65 61  §♣..D.A.....Ulea
0012FC14  64 20 43 44 20 26 20 44 56 44 20 50 69 63 74 75  d CD & DVD Pictu
0012FC24  72 65 53 68 6F 77 00 00 00 00 00 00 00 00 00 00  reShow..........
0012FC34  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0012FC44  00 00 00 00 00 00 00 00 00 00 00 00 55 6C 65 61  ............Ulea
0012FC54  64 20 43 44 20 26 20 44 56 44 20 50 69 63 74 75  d CD & DVD Pictu
0012FC64  72 65 53 68 6F 77 00 00 00 00 00 00 00 00 00 00  reShow..........
```

como vemos que luego se compara con ***579*** lo modificamos a ese valor directamente:

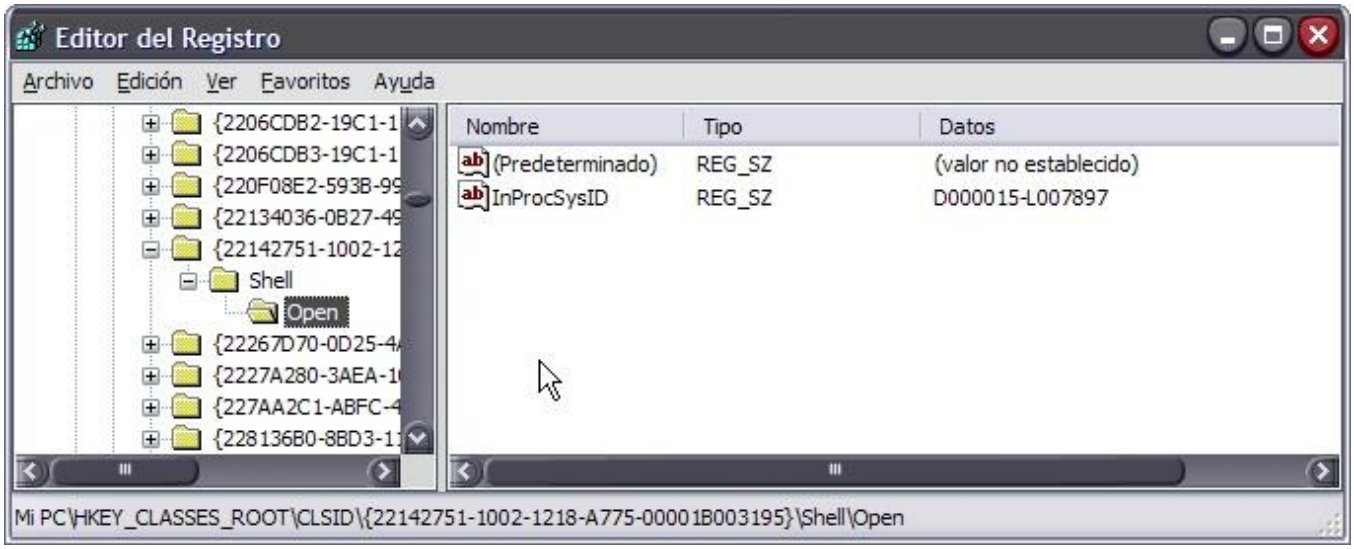
```
Address   Hex dump                                          ASCII
0012FC04  79 05 00 00 44 09 41 00 00 00 00 00 55 6C 65 61  y♣..D.A.....Ulea
0012FC14  64 20 43 44 20 26 20 44 56 44 20 50 69 63 74 75  d CD & DVD Pictu
0012FC24  72 65 53 68 6F 77 00 00 00 00 00 00 00 00 00 00  reShow..........
0012FC34  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0012FC44  00 00 00 00 00 00 00 00 00 00 00 00 55 6C 65 61  ............Ulea
0012FC54  64 20 43 44 20 26 20 44 56 44 20 50 69 63 74 75  d CD & DVD Pictu
0012FC64  72 65 53 68 6F 77 00 00 00 00 00 00 00 00 00 00  reShow..........
```

Ahora no llegamos al call que muestra la nag pero en el call de debajo en ***:17B48AC*** nos saca este regalito...

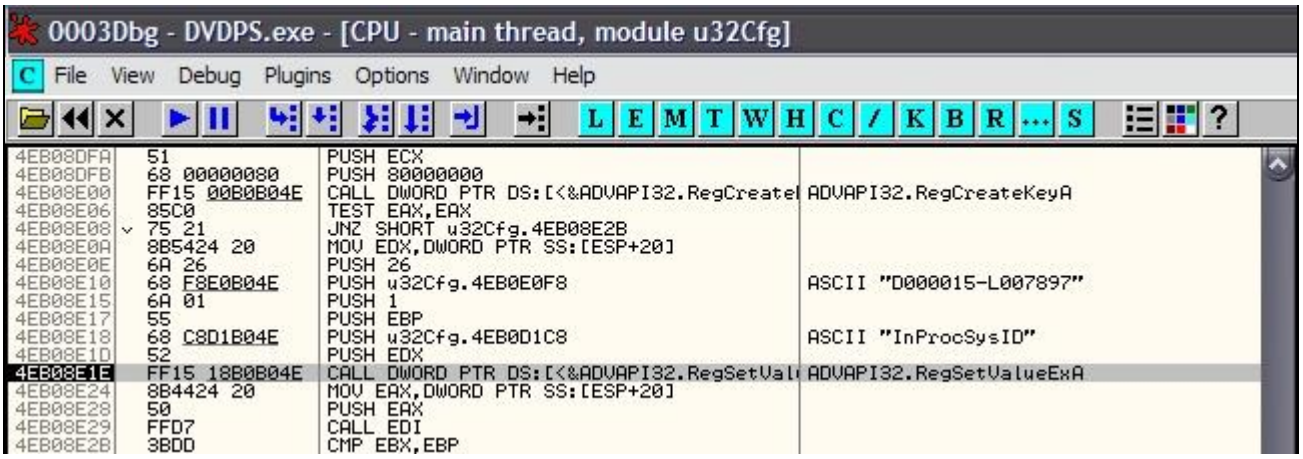
Esto nos hace ver que debemos entrar en el ***call ulcCheckLegality***, pues nada ponemos un ***HE*** en él y reiniciamos, vemos además que esta función pertenece a la librería ***u32Cfg.dll***, veamos los registros una vez allí:

```
Registers (FPU)
EAX 00000000
ECX FFFFFFFF
EDX 0012F6CC ASCII "CLSID\"
EBX 00000000
ESP 0012F564
EBP 0012FDC8
ESI 004106F8 DVDPS.004106F8
EDI 0012FADC ASCII "{22142751-1002-1218-A775-00001B003195}"
EIP 4EB08382 u32Cfg.4EB08382
```

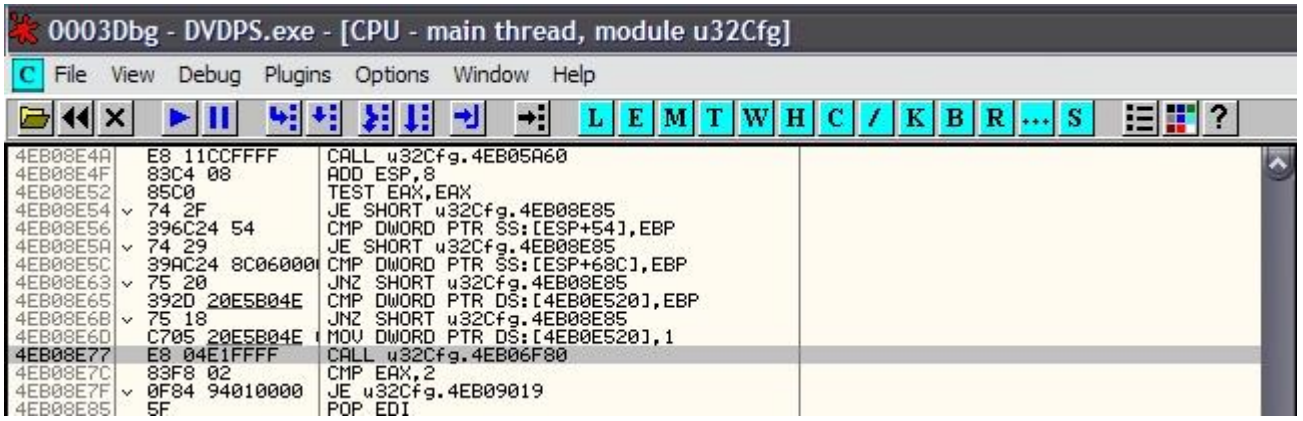
Queda claro que va a acceder al registro de Windowx, veamos que hay en la clave que busca:



El valor de ***InProcSysID*** parece buscar pues no hay nada más en esta clave, sigamos traceando a ver que más hace, llegamos a la zona donde manipula la cadena y después escribe en ella:



Sin comentarios no? sigamos...



No hemos traceado ni ***15*** instrucciones y nos topamos con la nag, está claro que entre el ***CALL EDI*** (Call ***RegCloseKey***) en ***:4EB08E29*** y el call de la nag se ha validado lo que sea del valor ***InProcSysID*** que fue a buscar al registro, la zona no es muy grande entre medio ya digo que no hay más de ***15*** ó ***20*** instrucciones así que el rastreo no debe ser difícil, a unas malas optamos por el parcheo del call y a volar se ha dicho ;XDDD

Borremos el ***HE*** anterior y pongámoslo en ***:4EB08E29*** que es donde cierra la clave a ver si descubrimos lo que quiere encontrar antes de optar por el parche ¿ok? en esta zona no me extenderé mucho, más bien nada porque no

tiene misterio, ah! otra cosa más cuando sale la nag y pulsamos ***Try Now!*** el retorno es ***EAX=132***, esto no está mal saberlo, voy a destacar que las comprobaciones de tiempo, o sea del Time-Trial, se realizan en ***:4EB08E4A***, hay una llamada a ***GetLocalTime*** que pone la fecha en los registros y se realizan las comprobaciones pertinentes, luego llegamos a esta zona en la que va a acceder a un archivo oculto que nos metió en el directorio de ***Windowz***:

```
0003Dbg - DVDPS.exe - [CPU - main thread, module u32Cfg]
C File View Debug Plugins Options Window Help
4EB05E80  51            PUSH ECX
4EB05E81  68 20D5B04E   PUSH u32Cfg.4EB0D520
4EB05E86  68 28D2B04E   PUSH u32Cfg.4EB0D228                    ASCII "USED"
4EB05E8B  68 5CD2B04E   PUSH u32Cfg.4EB0D25C                    ASCII "CHECKRUN"
4EB05E90  FF15 CCB0B04E CALL DWORD PTR DS:[<&KERNEL32.GetPrivat kernel32.GetPrivateProfileStringA
4EB05E96  8D5424 1C     LEA EDX,DWORD PTR SS:[ESP+1C]
4EB05E9A  8D8424 2803000 LEA EAX,DWORD PTR SS:[ESP+328]
```

Vemos como va a acceder a un archivo, la llamada a ***GetPrivateProfileStringA*** así lo delata, si miramos en los registros encontramos la información complementaria, jeje:

```
Registers (MMX)
EAX 0012EF14 ASCII "C:\WINDOWS\UPCOMM.MTX"
ECX 0012EE10
EDX 0012F122
EBX 00000000
ESP 0012EDDC
EBP 00000000
ESI 00000008
EDI 00000008

EIP 4EB05E90 u32Cfg.4EB05E90

C 0  ES 0023 32bit 0(FFFFFFFF)
```

Sí señor, ahí lo tenemos, guarda su info en este archivo, recordad de tener configuración para ver los archivos ocultos y de sistema (por si acaso) sino no lo podréis ver, está allí y contiene esta info:

[CHECKRUN]
VERSION=1070730
TECHID=1074510
BEGIN=2016227
USED=0000001

No es mucho pero sí suficiente, se está preparando para ver si sacar la nag diciendo ***Expired*** o ***Trial***, lo está verificando.!! lo más significativo es ***BEGIN*** que indica el año de instalación (2016 en mi caso, es que mi reloj está allí, jeje.. cosas que pasan), luego el día (22 en mi caso) y el mes (7 en mi caso) de ahí sale el ***2016227*** que vemos, también ***USED***, que marca el número de días que pasaron desde que se instaló el programa, echemos un vistazo al stack a ver con que argumentos se llama a la función:

```
0012EDDC  4EB0D25C  Section = "CHECKRUN"
0012EDE0  4EB0D228  Key = "USED"
0012EDE4  4EB0D520  Default = ""
0012EDE8  0012EE10  ReturnBuffer = 0012EE10
0012EDEC  00000104  BufSize = 104 (260.)
0012EDF0  0012EF14  IniFileName = "C:\WINDOWS\UPCOMM.MTX"
0012EDF4  77DA17D8  ADVAPI32.RegCloseKey
```

Lo que ya sabemos, dice que ***:12EE10*** hará las veces de buffer, ***Follow in Dump*** y lo ponemos a la vista, así veremos la información que recibe, es esta:

```
Address   Hex dump                                          ASCII
0012EE10  30 30 30 30 30 30 31 00 00 00 00 00 00 00 00 00  0000001.........
0012EE20  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0012EE30  00 00 00 00 00 00 00 00 00 00 00 00 75 00 00 00  ............u...
0012EE40  48 F5 12 00 E4 EE 12 00 00 00 00 00 00 00 00 00  H§↨.♂¯↨.........
0012EE50  00 00 00 00 00 02 01 00 01 00 00 00 00 00 00 00  .....☻☺.☺.......
```

Recibió el ***USED*** (número de días) que en nuestro caso es ***0000001*** porque lo acabamos de instalar, supongo que luego cogerá este número y lo comparará con el límite aunque eso parece que ya lo hizo antes mediante la llamada a ***GetLocalTime***, por si acaso pongamos un ***BPM On Memory Acces*** en el valor recogido a ver donde para, pero si no hace ni falta si justo después hace una comparación ya, veamos donde:

```
0003Dbg - DVDPS.exe - [CPU - main thread, module u32Cfg]
File View Debug Plugins Options Window Help
4EB05E96   8D5424 1C        LEA EDX,DWORD PTR SS:[ESP+1C]
4EB05E9A   8D8424 2803000(  LEA EAX,DWORD PTR SS:[ESP+328]
4EB05EA1   52               PUSH EDX
4EB05EA2   50               PUSH EAX
4EB05EA3   FF15 64B1B04E    CALL DWORD PTR DS:[<&MSVCRT._mbscmp>]   msvcrt._mbscmp
4EB05EA9   83C4 08          ADD ESP,8
4EB05EAC   85C0             TEST EAX,EAX
4EB05EAE   74 20            JE SHORT u32Cfg.4EB05ED0
```

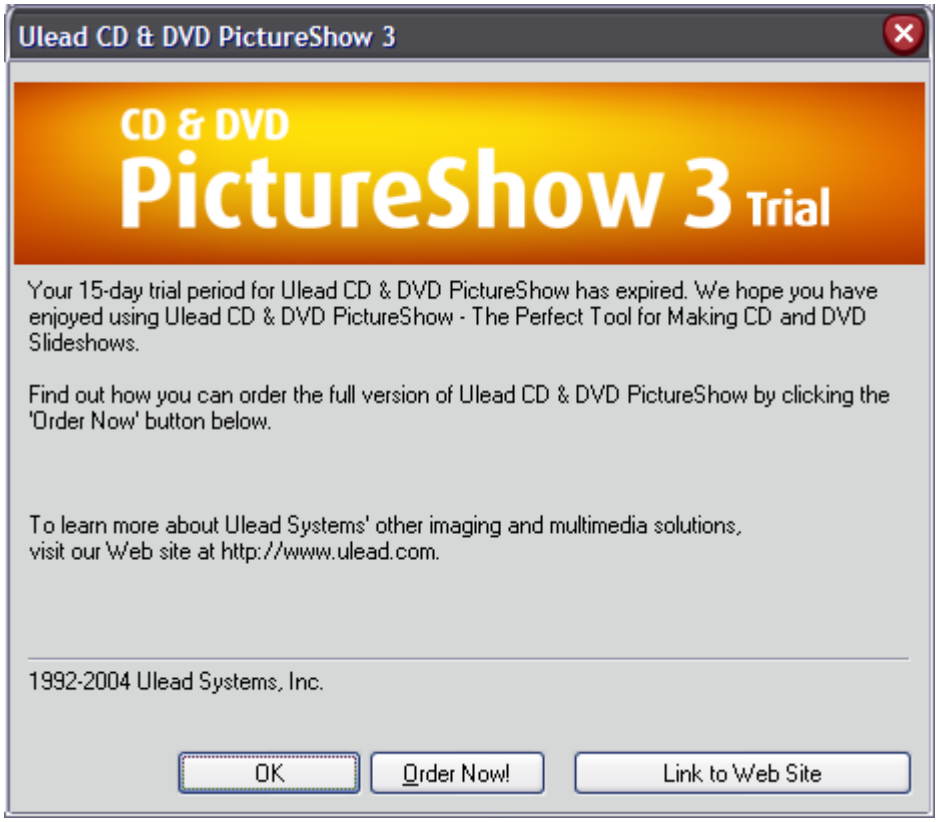
Aquí recoge el valor obtenido en la instrucción '***LEA EAX, DWORD PTR SS:[ESP+328]***' donde pone en ***EAX*** el ***0000001*** que recogió de nuestro archivo y parece que lo va a comparar en el siguiente call, veamos los registros:

```
Registers (MMX)
EAX 0012F11C ASCII "0000007"
ECX 77F4166A ntdll.77F4166A
EDX 0012EE10 ASCII "0000001"
EBX 00000000
ESP 0012EDEC
EBP 00000000
ESI 00000008
EDI 00000008
EIP 4EB05EA3 u32Cfg.4EB05EA3
```

Más claro agua, lo compara con ***0000007*** ¿pero el límite de días no son ***14***? para qué lo comparará con ***7***? bueno sigamos un poco a ver que más hace, la llamada retorna ***EAX=1*** y no saltará en :4EB05EAE, llegamos a una llamada a ***WritePrivateProfileStringA***, donde cambiará el valor de ***0000001*** a ***0000007***, si sólo lo hemos usado un día, qué cabr...!! nos ha chorizado 6 DÍAS...!!! adjunto el stack para que no haya dudas:

```
0012EDE4   4EB0D25C   Section = "CHECKRUN"
0012EDE8   4EB0D228   Key = "USED"
0012EDEC   0012F11C   String = "0000007"
0012EDF0   0012EF14   FileName = "C:\WINDOWS\UPCOMM.MTX"
0012EDF4   77DA17D8   ADVAPI32.RegCloseKey
```

Se ve claro no? ya sabemos como evitar que se modifique el archivo, jeje.. pero sigamos, Hostias! a la salida del call salta por encima de la nag (el salto se produce en :4EB08E3E), sigamos ¿Queeeeeeé?



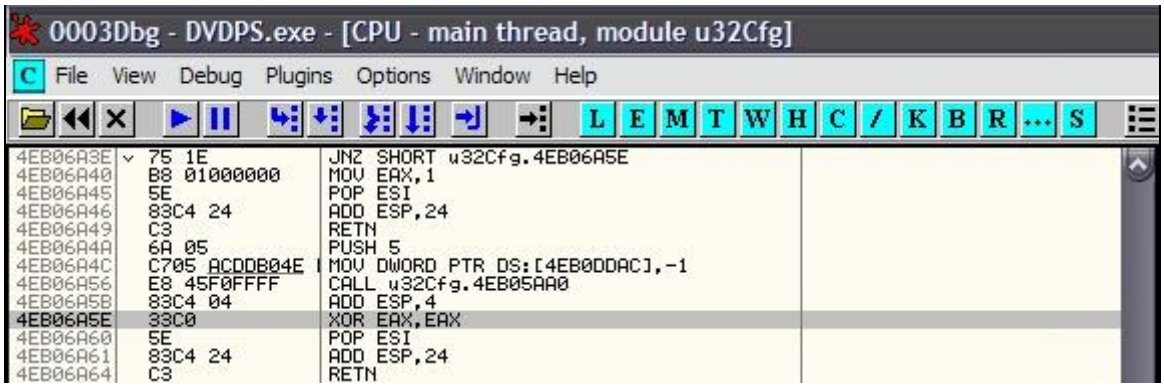
Pero como que ***EXPIRED***? si no hemos tocado nada ¿nos ha detectado? seguramente porque os aseguro que no he tocado nada y lo acabo de instalar, hijo p... bueno miremos el lado positivo, sabemos que el salto en ***:4EB08E3E*** no es muy bueno que se diga, el call de la nag está es esta parte:

```
:4EB08EC6   E8 E5E0FFFF      CALL u32Cfg.4EB06FB0        ; Call_Nag_Expired..!!
:4EB08ECB   3D 33010000      CMP EAX,133
:4EB08ED0   0F85 43010000    JNZ u32Cfg.4EB09019
```

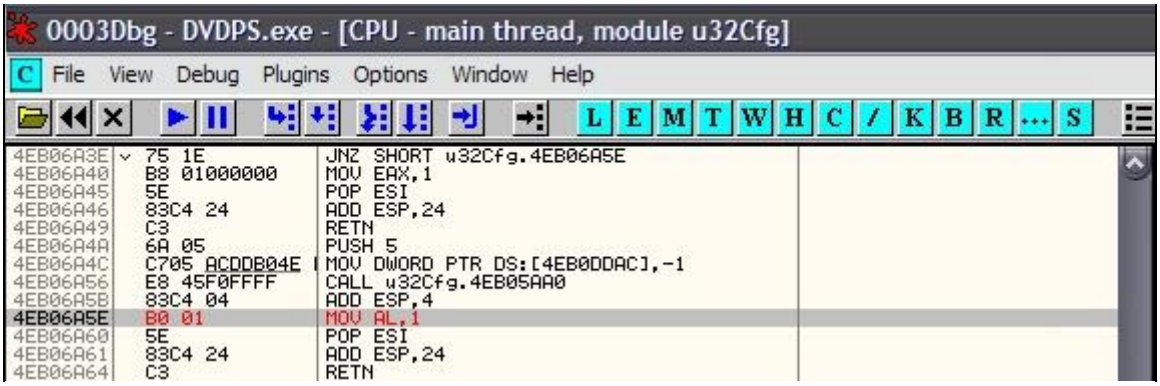
Pues vaya como se las gasta, pero bueno tenemos datos más que interesante, además ya sabemos donde esconde las dos ***nags*** ;XDD (a no ser que tenga más claro), entonces sabemos que:

```
:4EB08E34   E8 A7DAFFFF      CALL u32Cfg.4EB068E0        ; Verifica USED, debe retornar EAX=!0
:4EB08E39   83C4 04          ADD ESP,4
:4EB08E3C   85C0             TEST EAX,EAX
:4EB08E3E   74 55            JE SHORT u32Cfg.4EB08E95    ; No debe saltar
```

Para que esto sea así le cambiaremos el retorno de esta manera:



que cambiaremos por:



Y ya salimos con ***EAX=1***, jeje.. pero con esto veríamos la ***nag_trial*** cada vez, intentemos evitarla también, si miramos la imagen del ***call_nag_trial*** vemos que encima hay un salto (en ***:4EB08E54***) que de producirse salta por encima, este salto lo controla el call anterior, entraremos y lo obligaremos a retornar ***EAX=0***, un comentario, ahora se verá en la imagen siguiente, hay una llamada a la función ***_atoi*** de la librería ***mscvrt***, esta debe retornar ***EAX= -1***, ya que luego hay un ***TEST EAX, EAX*** seguido de un ***SETGE DL*** que hará ***DL=1*** en caso de que el valor de ***EAX*** sea superior o igual a 0, como queremos ***DL=0*** esto no debe ocurrir como consecuencia necesitaremos que retorne ***EAX= -1*** ¿ok? otra posibilidad más sencilla es modificar el código un poco y simular el mismo resultado que es lo que haremos aquí, fijaos que se guarda el retorno (que lo lleva ***EAX***) en ***[ECX]*** después de tomar su valor ***EAX*** de ***DL***, esto se solventaría fácil con un par de retoques, os mostraré una imagen del antes y el después de la zona, es la manera más fácil que encontré de salir con ***EAX=0*** del call, notad que he invertido un par de instrucciones porque así me va mejor, veamos las imágenes:

```
0003Dbg - DVDPS.exe - [CPU - main thread, module u32Cfg]
4EB05A80   52            PUSH EDX
4EB05A81   FF15 54B1B04E CALL DWORD PTR DS:[<&MSVCRT.atoi>]      msvcrt.atoi
4EB05A87   8B4C24 14     MOV ECX,DWORD PTR SS:[ESP+14]
4EB05A8B   33D2          XOR EDX,EDX
4EB05A8D   85C0          TEST EAX,EAX
4EB05A8F   0F9DC2        SETGE DL
4EB05A92   8901          MOV DWORD PTR DS:[ECX],EAX
4EB05A94   8BC2          MOV EAX,EDX
4EB05A96   83C4 0C       ADD ESP,0C
4EB05A99   C3            RETN
```

Este es el antes de la zona, veamos el después:

```
0003Dbg - DVDPS.exe - [CPU - main thread, module u32Cfg]
4EB05A80   52            PUSH EDX
4EB05A81   FF15 54B1B04E CALL DWORD PTR DS:[<&MSVCRT.atoi>]      msvcrt.atoi
4EB05A87   8B4C24 14     MOV ECX,DWORD PTR SS:[ESP+14]
4EB05A8B   33D2          XOR EDX,EDX
4EB05A8D   85C0          TEST EAX,EAX
4EB05A8F   B2 00         MOV DL,0
4EB05A91   90            NOP
4EB05A92   8BC2          MOV EAX,EDX
4EB05A94   8901          MOV DWORD PTR DS:[ECX],EAX
4EB05A96   83C4 0C       ADD ESP,0C
4EB05A99   C3            RETN
```

Lo que está en rojo es lo que hemos modificado, salvemos los cambios y probemos el archivo resultante, notad que al ser una la ***dll*** debemos salvar con otro nombre, cerrar el programa y renombrar la original a otro nombre y la del parche con el nombre original y entonces probar los resultados.. Pues no sale la nag pero el programa tampoco ¿?

Carguémoslo en ***Olly*** y a tracear pues parece que nos pillaron, no importa, para ver como vamos dejamos la dll original en su sitio, queremos ver como vamos, además vamos al archivo que vimos en el directorio de windowx y le ponemos un ***0000001*** en ***USED*** nos interesa ver donde va a sacar el programa, hecho esto (para no poner un millón de imágenes) os comento que sale el baner pero no la aplicación, es más en la nag ponía ***14 days left*** o sea que sin duda alguna nos detectaron, bueno, miremos esta zona un momento:

```
0003Dbg - DVDPS.exe - [CPU - main thread, module DVDPS]
00401538  .  FF15 2C06410( CALL DWORD PTR DS:[41062C]      PS2Base.?m_fnInitProdInfo@OEM_INFO@@QAEHXZ
00401540  .  85C0          TEST EAX,EAX
00401542  .v 75 04         JNZ SHORT DVDPS.00401548
00401544  .  33C0          XOR EAX,EAX
00401546  .v EB 46         JMP SHORT DVDPS.0040158E
00401548  >  8B4D F4       MOV ECX,DWORD PTR SS:[EBP-C]
0040154B  .  E8 45130000   CALL DVDPS.00402895             call que muestra en baner..!!
00401550  .  8B4D F4       MOV ECX,DWORD PTR SS:[EBP-C]
00401553  .  E8 3A000000   CALL DVDPS.00401592             Interesante llamada..!!
00401558  .  8945 F8       MOV DWORD PTR SS:[EBP-8],EAX
0040155B  .  8B4D F4       MOV ECX,DWORD PTR SS:[EBP-C]
0040155E  .  E8 C6120000   CALL DVDPS.00402829
```

Esta es la zona clave, en ***:40153A*** se realizan las comprobaciones pertinentes, si todo va bien saltará en ***:401542***, llegamos a ***:40154B*** donde saca el baner y la más importante en ***:401553*** donde está puesto el último ***BPX***, esta llamada se encarga de inicializar la aplicación, cargar todas las librerías que necesita y en fin, que es el call más importante de todos los que vemos, por eso para encontrar el error debemos entrar dentro, y allí estudiar a ver que hace, como supongo que todos sabemos mirar no voy a poner muchas más fotos sino al final el archivo pesará más que el exe y no es manera, solo comentaré y en todo caso alguna imagen esporádica si es necesaria y la ocasión lo requiere.

Una vez que paremos sobre el call entraremos como siempre pulsando ***F7*** una vez, a partir de aquí ya es traceo y vista, ojo a los registros y al stack (***pila***) la ventana dump puede ser interesante también no la descartemos, pues eso vamos traceando con ***F8*** y vemos en la barra de estado de ***Olly*** como en algunos subcalls se van cargando ***dlls***

que necesita el programa, pondré una imagen aquí porque necesito mostrar un salto que no nos interesa, esto es importante porque todos los siguientes saltos que apunten aquí sabemos que no son muy buenos para nosotros:

```
0003Dbg - DVDPS.exe - [CPU - main thread, module DVDPS]
004015B6  .   E8 07190000   CALL DVDPS.00402EC2
004015BB  .   85C0          TEST EAX,EAX
004015BD  .v  75 07         JNZ SHORT DVDPS.004015C6
004015BF  .   33C0          XOR EAX,EAX
004015C1  .v  E9 7D030000   JMP DVDPS.00401943
004015C6  >   8D4D D4       LEA ECX,DWORD PTR SS:[EBP-2C]
```

El ***call 402EC2*** carga algunas librerías que necesita el programa, a su salida es ***EAX=1*** y salta en ***:4015BD***, lo interesante aquí es ver que de no haberse producido el salto porque ***EAX=0*** hubiéramos llegado al ***JMP 401943*** que nos hubiera sacado de la rutina, pues este detalle es muy importante porque a partir de ahora y hasta el siguiente ***RET*** todos los saltos que apunten a esta dirección serán sospechosos y debemos desconfiar ¿entendido? ahora sigamos traceando, entramos en un loop pero poco rato salimos y llegamos a este call, atención:

```
0003Dbg - DVDPS.exe - [CPU - main thread, module DVDPS]
004016B6  .   FF52 14       CALL DWORD PTR DS:[EDX+14]
004016B9  .   85C0          TEST EAX,EAX
004016BB  .v  75 1E         JNZ SHORT DVDPS.004016DB
004016BD  .   C745 B8 0000  MOV DWORD PTR SS:[EBP-48],0
004016C4  .   C745 FC FFFF  MOV DWORD PTR SS:[EBP-4],-1
004016CB  .   8D4D D4       LEA ECX,DWORD PTR SS:[EBP-2C]
004016CE  .   E8 F9860000   CALL <JMP.&MFC42.#800>
004016D3  .   8B45 B8       MOV EAX,DWORD PTR SS:[EBP-48]
004016D6  .v  E9 68020000   JMP DVDPS.00401943
004016DB  >   6A 00         PUSH 0
004016DD  .   FF15 B806410  CALL DWORD PTR DS:[4106B8]      DVDPS.0040A60E
```

Llegamos a otra salto hacia :***401943*** pero lo evitamos, perfecto sigamos... CUIDADO A ESTO..!!

```
0003Dbg - DVDPS.exe - [CPU - main thread, module DVDPS]
00401738  .   E8 270C0000   CALL DVDPS.00402364             DVDPS.00402364
0040173D  .   8945 D8       MOV DWORD PTR SS:[EBP-28],EAX
00401740  .   837D D8 00    CMP DWORD PTR SS:[EBP-28],0
00401744  .v  75 1E         JNZ SHORT DVDPS.00401764
00401746  .   C745 B0 0000  MOV DWORD PTR SS:[EBP-50],0
0040174D  .   C745 FC FFFF  MOV DWORD PTR SS:[EBP-4],-1
00401754  .   8D4D D4       LEA ECX,DWORD PTR SS:[EBP-2C]
00401757  .   E8 70860000   CALL <JMP.&MFC42.#800>
0040175C  .   8B45 B0       MOV EAX,DWORD PTR SS:[EBP-50]
0040175F  .v  E9 DF010000   JMP DVDPS.00401943
00401764  >   FF15 1806410  CALL DWORD PTR DS:[410618]      DVDPS.00409A66
0040176A  .   8BC8          MOV ECX,EAX
```

Ahora no va a saltar y llegaremos a un salto de esos que no nos gustan ya que nos tiran fuera, cambiemos el salto:

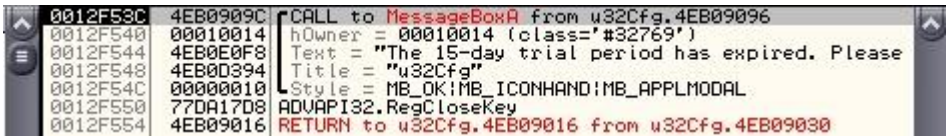
```
0003Dbg - DVDPS.exe - [CPU - main thread, module DVDPS]
00401738  .   E8 270C0000   CALL DVDPS.00402364             DVDPS.00402364
0040173D  .   8945 D8       MOV DWORD PTR SS:[EBP-28],EAX
00401740  .   837D D8 00    CMP DWORD PTR SS:[EBP-28],0
00401744  .v  EB 1E         JMP SHORT DVDPS.00401764
00401746  .   C745 B0 0000  MOV DWORD PTR SS:[EBP-50],0
0040174D  .   C745 FC FFFF  MOV DWORD PTR SS:[EBP-4],-1
00401754  .   8D4D D4       LEA ECX,DWORD PTR SS:[EBP-2C]
00401757  .   E8 70860000   CALL <JMP.&MFC42.#800>
0040175C  .   8B45 B0       MOV EAX,DWORD PTR SS:[EBP-50]
0040175F  .v  E9 DF010000   JMP DVDPS.00401943
00401764  >   FF15 1806410  CALL DWORD PTR DS:[410618]      DVDPS.00409A66
0040176A  .   8BC8          MOV ECX,EAX
0040176C  .   FF15 1C06410  CALL DWORD PTR DS:[41061C]      DVDPS.00409A54
00401772  .   FF15 6806410  CALL DWORD PTR DS:[410668]      DVDPS.00409988
```

Hemos cambiado el salto, lo hemos ***jumpeado*** para que salte por encima del "***JMP 401943"*** que no nos gusta, ahora sigamos con sumo cuidado evitando estos saltos si los hubiera, vamos pasando por algunos calls que siguen cargando dll's esto es bueno, llegamos al ***RET*** sin problemas le damos a ***F9*** y... ***SIIIIIIIIIII***..!! Sale el programa, jeje.. le hemos pillado, salvemos el cambio y tratemos de correr el parche con la dll parcheada también (ya sabéis, cambiarle el nombre a la original y renombrar la parcheada con el propio nombre original sino no la encontrará y dirá que le falta una ***dll*** y no funkará), hacemos esto y NO FUNCIONA..!!

Cambiar el nombre del archivo (el .exe) parcheado por el nombre original... FUNCIONA..!!

Ahora sí tenemos el crack del programa, adelantemos el reloj 2 años.. jeje..

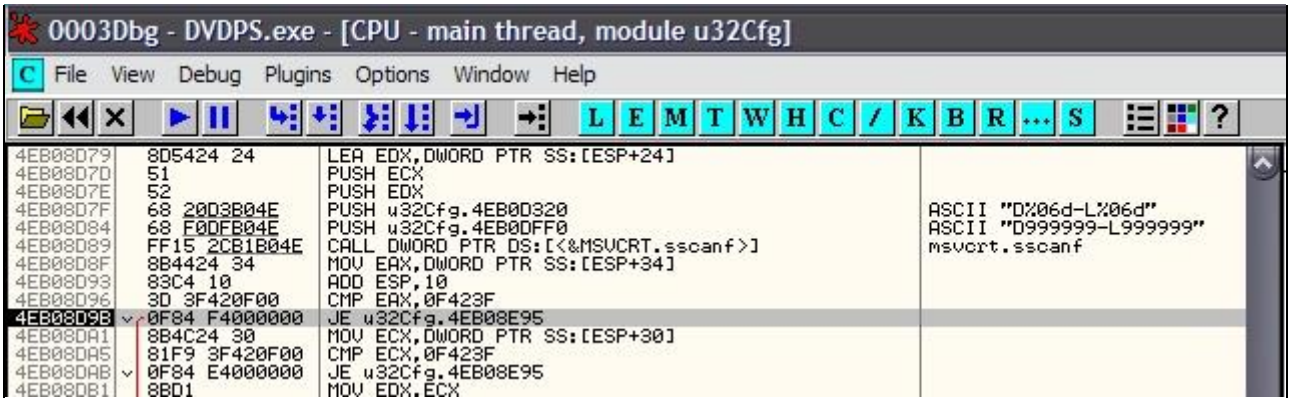
Otro problema, la limitación temporal sigue en pié, a por ella... cargamos el programa en ***Olly*** y ponemos un ***BPX*** en la api ***MessageBoxA*** desde el ***CommandBar*** (ya sabréis como supongo). Cuando rompa ***Olly*** ojo al stack:



La llamada se realizó desde ***:4EB0909C***, está claro que está en ***u32Cfg.dll*** como se puede ver en la imagen, esto no nos va a detener, retornemos del call con ***Ctrl+F9***, aparecerá en mensaje en pantalla, lo aceptamos y caemos en el ***RET***, lo superamos y ya estamos debajo del call responsable, busquemos un salto que lo evite, como no lo vemos ***Ctrl+F9*** otra vez así hasta llegar a esta zona:

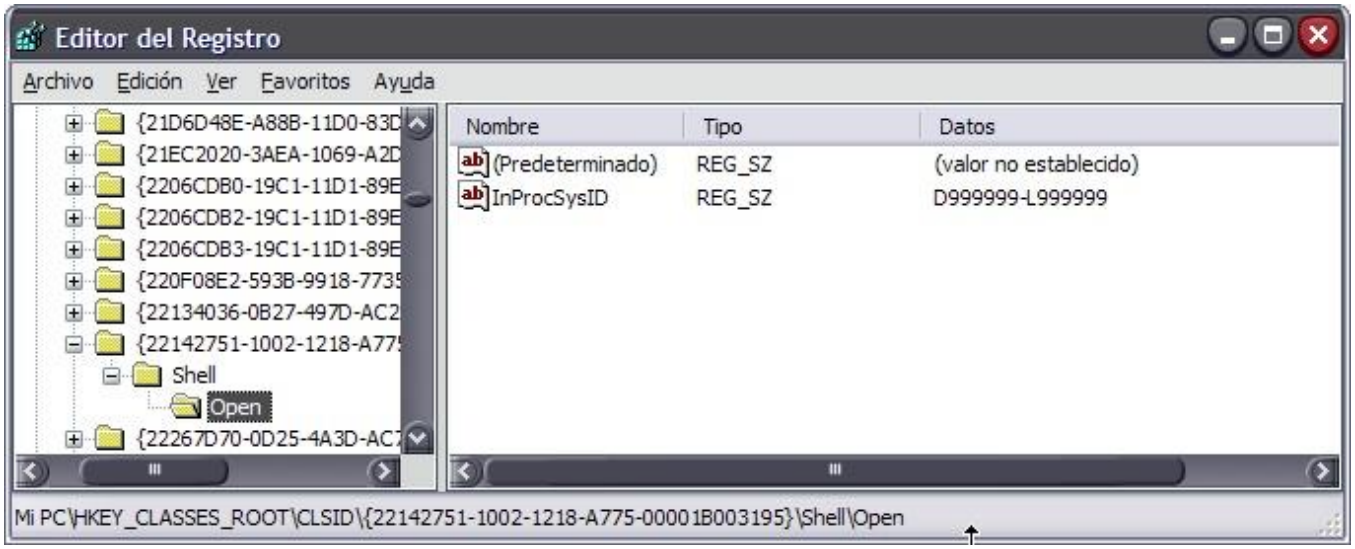
```
:017B488B   E8 C8B30100      CALL <JMP.&u32Cfg.ulcCheckLegality>
:017B4890   85C0             TEST EAX,EAX
:017B4892   75 07            JNZ SHORT PS2Base.017B489B
```

Conocida verdad? pongamos un ***BPHE*** en ***:017B488B*** y reiniciemos con ***Ctrl+F2***, luego ***F9*** para parar allí, entremos con ***F7*** y a tracear como siempre, llegamos a unas llamadas muy conocidas pero en otro lugar:



En la imagen no se ve pero esta verificando el valor del valor ***InProcSysID*** que tiene en el registro, a la salida del call ***MSVCRT.sscanf*** vemos que ***EAX=0F423F*** justo el valor que verifica justo antes del salto en el que estamos, veamos que significa este valor:

Ahí está la respuesta en el ***CommandBar***, como se ve seleccionado ***"0F423Fh = 999999d"*** veamos cuanto vale el valor que tiene metido en el registro y al que acaba de acceder:



Pues se confirman nuestras sospechas, parece ser que este valor es el que le está diciendo al programa que nuestro tiempo de evaluación ha terminado. Aquí las variantes de ataque son múltiples, optaremos por una fácil, sabemos el lugar en que accede al registro a leer la información:

```
:4EB08D89   FF15 2CB1B04E   CALL DWORD PTR DS:[<&MSVCRT.sscanf>]
:4EB08D8F   8B4424 34       MOV EAX,DWORD PTR SS:[ESP+34]
:4EB08D93   83C4 10         ADD ESP,10
:4EB08D96   3D 3F420F00     CMP EAX,0F423F              ;Si salta es que ha expirado.!!
:4EB08D9B   0F84 F4000000   JE u32Cfg.4EB08E95
:4EB08DA1   8B4C24 30       MOV ECX,DWORD PTR SS:[ESP+30]
:4EB08DA5   81F9 3F420F00   CMP ECX,0F423F
:4EB08DAB   0F84 E4000000   JE u32Cfg.4EB08E95          ;Si salta es que ha expirado.!!
. . .       . . .           . . .
:4EB08E00   FF15 00B0B04E   CALL DWORD PTR DS:[<&ADVAPI32.RegCreateKeyA>]    ;
:4EB08E06   85C0            TEST EAX,EAX
:4EB08E08   75 21           JNZ SHORT u32Cfg.4EB08E2B       ; Aquí debe saltar..!!
:4EB08E0A   8B5424 20       MOV EDX,DWORD PTR SS:[ESP+20]
:4EB08E0E   6A 26           PUSH 26
:4EB08E10   68 F8E0B04E     PUSH u32Cfg.4EB0E0F8            ; ASCII "D999999-L999999"
:4EB08E15   6A 01           PUSH 1
:4EB08E17   55              PUSH EBP
:4EB08E18   68 C8D1B04E     PUSH u32Cfg.4EB0D1C8            ; ASCII "InProcSysID"
:4EB08E1D   52              PUSH EDX
:4EB08E1E   FF15 18B0B04E   CALL DWORD PTR DS:[<&ADVAPI32.RegSetValueExA>]   ; Modifica el valor
. . .       . . .           . . .
:4EB08E29   FFD7            CALL EDI
:4EB08E2B   3BDD            CMP EBX,EBP
:4EB08E2D   75 66           JNZ SHORT u32Cfg.4EB08E95       ;Si salta es que ha expirado.!!
:4EB08E2F   8B4C24 24       MOV ECX,DWORD PTR SS:[ESP+24]
:4EB08E33   51              PUSH ECX
:4EB08E34   E8 A7DAFFFF     CALL u32Cfg.4EB068E0
:4EB08E39   83C4 04         ADD ESP,4
:4EB08E3C   85C0            TEST EAX,EAX
:4EB08E3E   74 55           JE SHORT u32Cfg.4EB08E95        ;Si salta es que ha expirado.!!
```

Estas son las comprobaciones que realiza del registro, lee el contenido y mira si es ***999999***, en este caso vamos a Expired de cabeza, si no lo verifica de nuevo pero ahora con ***ECX***, pero notad que recoge el valor de ***[ESP+30],*** pero de momento ignoraremos este detalle (ya veremos al final).

Vamos a saco, sabemos que los saltos hacia ***:4EB08E95*** son nefastos, pues los nopeamos todos, estos están en:

□ 4EB08D9B

□ 4EB08DAB
□ 4EB08E2D
□ 4EB08E3E

Todos ellos los nopeamos, además cambiaremos:

```
:4EB08E08    75 21    JNZ SHORT u32Cfg.4EB08E2B    POR:
:4EB08E08    EB 21    JMP SHORT u32Cfg.4EB08E2B
```

Para que no modifique más el valor "***InProcSysID***" del registro, y por último (este no es necesario)

```
:4EB08E54   74 2F    JE SHORT u32Cfg.4EB08E85    POR:
:4EB08E54   EB 2F    JMP SHORT u32Cfg.4EB08E85
```

Esto es para que no saque más la ***nag_Trial***, que aunque hemos parcheado el call nunca se sabe, jeje..

Como comentario decir que el programa va accediendo al registro aún cuando ya esta corriendo, probar de cambiar el valor de "***InProcSysID***" y ya veréis como ***Expira*** instantáneamente, además tiene un ***Checksum*** en esta clave pues según lo que toquemos nos sale un mensaje diciendo no se qué de la integridad de la información, etc.. o sea que se da cuenta que hemos tocado la clave.

Y con estos cambios si conseguimos burlar la seguridad del programa, ha sido más largo el tuto de lo que me hubiera gustado pero bueno, así lo ha requerido el programa ***;XDDD***

Y al fin hemos terminado el tuto de hoy, espero que de verdad hayáis disfrutado, yo lo he hecho y mucho, jeje.. para cualquier duda, pregunta, sugerencia, crítica o lo que queráis sentíos libres de escribirme a:

syxe00@yahoo.es
syxe00@hotmail.com

Por supuesto todos los mensajes serán contestados. Hasta la próxima